

Chương 2

Lớp và đối tượng

Nội dung

- 2.1 Giới thiệu lập trình hướng đối tượng
- 2.2 Lớp và các thành phần
- 2.3 Tạo lớp theo mô tả các thuộc tính và phương thức
- 2.4 Đối tượng
- 2.5 Nạp chồng hàm và toán tử
- 2.6 Thư viện chuẩn

2.1 Giới thiệu lập trình hướng đối tượng

Lập trình hướng đối tượng là phương pháp lập trình dựa trên các khái niệm về “đối tượng”, là các cấu trúc chứa cả dữ liệu và phương thức xử lý

2.2 Lớp và thành phần

- 2.1.1 Khai báo lớp
 - Lớp là một kiểu dữ liệu trừu tượng
 - Phản ánh một thực thể trong bài toán
 - Do người lập trình định nghĩa
 - Gồm 2 thành phần:
 - Thuộc tính: các thành phần mang dữ liệu
 - Phương thức: các hàm tương tác với các thuộc tính

2.2 Lớp và thành phần

- 2.1.1 Khai báo lớp

- Cú pháp khai báo:

```
class <tên lớp> {
```

```
private:
```

```
    <các thành phần riêng>
```

```
public:
```

```
    <các thành phần công cộng>
```

```
};
```

```
<định nghĩa các hàm thành phần chưa  
được định nghĩa bên trong lớp>
```

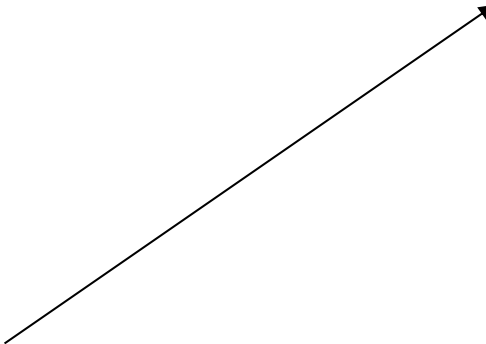
2.2 Lớp và thành phần

- 2.1.1 Khai báo lớp

VD:

```
class point {  
    private:  
        int x, y;  
    public:  
        void init(int ox, int oy);  
        void move(int dx, int dy);  
        void display();  
};
```

• Các thuộc tính: 2 biến thành phần x, y



• Các phương thức:
3 hàm thành phần
(nguyên mẫu)



Định nghĩa hàm thành phần

```
void point::init(int ox, int oy) {  
    cout<<"ham thanh phan intit \n";  
    x=ox; y=oy;  
}  
  
void point::move(int dx, int dy) {  
    cout<<"ham thanh phan move \n";  
    x+=dx; y+=dy;  
}  
  
void point::display() {  
    cout<<"ham thanh phan display \n";  
    cout<<"toa do : "<<x<<" , " <<y<<"\n";  
}
```

2.2 Lớp và thành phần

- 2.2.2 Thuộc tính truy cập

- **Private:**

- Các thành phần sau từ khóa này được bảo vệ và không thể truy cập từ bên ngoài, chỉ có thể truy cập bởi các hàm thành phần của lớp

- **Public:**

- Các thành phần được phép truy cập từ bất kỳ hàm nào

- **Protected**

- Tương tự private, nhưng cho phép hàm thành phần lớp con có thể truy cập.

2.2 Lớp và thành phần

- 2.2.3 Dữ liệu thành phần

- Còn gọi là thuộc tính (property) của lớp
- Là các biến dữ liệu khai báo bên trong lớp
- Thể hiện các dữ liệu mà một lớp phản ánh
- Cú pháp khai báo:

`<kiểu> <tên biến>;`

- Kiểu của dữ liệu có thể là bất kỳ kiểu nào, kể cả là chính lớp đang khai báo.

2.2 Lớp và thành phần

- 2.2.4 Hàm thành phần
 - Còn gọi là phương thức (method) của lớp
 - Là các hàm được khai báo bên trong lớp
 - Xử lý dữ liệu bên trong và bên ngoài lớp
 - Các loại hàm của lớp:
 - Hàm thông thường
 - Hàm tạo
 - Hàm hủy
 - Hàm toán tử

2.2 Lớp và thành phần

- 2.2.4 Hàm thành phần

- Khai báo hàm thành phần:

- Cách 1: Khai báo và định nghĩa hàm thành phần bên trong lớp

```
<kiểu trả về> <tên hàm> (<danh sách tham số>) {  
    // các lệnh trong thân hàm  
    [return giá_trị_trả_về;]  
}
```

2.2 Lớp và thành phần

- 2.2.2 Hàm thành phần

- Khai báo hàm thành phần:

- Cách 2: Khai báo nguyên mẫu hàm bên trong lớp, định nghĩa hàm bên ngoài lớp:

- Bên trong lớp :

- `<kiểu trả về> <tên hàm>(<danh sách tham số>);`

- Bên ngoài lớp:

- `<kiểu trả về> <tên lớp>::<tên hàm>(<danh sách tham số>){`
 `<thân hàm>`
 `}`

2.2 Lớp và thành phần

- 2.2.5 Hàm tạo (hàm thiết lập):
 - Là một hàm thành phần đặc biệt dùng để khởi tạo giá trị trong lớp và thực hiện các công việc chuẩn bị khác.
 - Là hàm không thể thiếu, ngay cả khi người lập trình không định nghĩa thì chương trình cũng sẽ tự tạo một hàm thiết lập ngầm định cho lớp.
 - Tự động được gọi khi khai báo đối tượng.
 - Có thể có tham số hoặc không có tham số
 - Tên hàm thiết lập phải trùng tên lớp và không có kiểu trả về.
 - Phải có thuộc tính truy cập là public

2.2 Lớp và thành phần

- 2.2.5 Hàm tạo (hàm thiết lập):

```
class point{  
    int x,y;  
public:  
    point(int ox,int oy){  
        x=ox;y=oy;  
    }  
}
```

```
int main{  
    point p1(2,3); //x=2,y=3  
    return 0;  
}
```

2.2 Lớp và thành phần

- 2.2.5 Hàm tạo (hàm thiết lập):
 - Hàm thiết lập ngầm định:
 - Là hàm thiết lập được tự động sinh ra nếu người dùng không định nghĩa hàm thiết lập nào.
 - Là hàm thiết lập không có tham số hoặc tham số nhận giá trị ngầm định
 - Bắt buộc phải có trong trường hợp khai báo mảng đối tượng
 - VD: `point array_point[10];`

- VD

2.2 Lớp và thành phần

- 2.2.5 Hàm tạo (hàm thiết lập):
 - Hàm thiết lập sao chép:
 - Dùng để sao chép dữ liệu từ đối tượng này sang đối tượng mới khi khởi tạo đối tượng.
 - Nếu người dùng không định nghĩa, chương trình tự tạo ra hàm thiết lập sao chép ngầm định.
 - Cú pháp khai báo: `<tên lớp>([const] <tên lớp>&);`
 - Tự động được gọi khi gặp 1 trong hai câu lệnh sau:
 - `Point a2=a1;`
 - `Point a2(a1)`

2.2 Lớp và thành phần

- 2.2.6 Hàm hủy :

- Là một hàm đặc biệt của lớp, có chức năng ngược với hàm thiết lập
- Thực hiện các công việc trước khi hủy một đối tượng: giải phóng các vùng nhớ, xóa đối tượng khỏi màn hình, in thông báo,...
- Không có kiểu trả về
- Tên bắt đầu bằng dấu ~ và tên lớp
- Không có tham số

2.3 Tạo lớp theo mô tả về thuộc tính và phương thức

- Xác định thực thể -> tên lớp
- Xác định các dữ liệu gắn với thực thể -> thuộc tính của lớp -> tên và kiểu dữ liệu của các thuộc tính
- Xác định các hành động, hoạt động của thực thể -> phương thức (hàm) của lớp -> tên, tham số, nội dung của các phương thức
- Cách tạo đối tượng từ lớp -> hàm khởi tạo

2.4 Đối tượng

- 2.4.1 Khai báo đối tượng:
 - Đối tượng là một thể hiện của một lớp khi thực thi chương trình.
 - Các thuộc tính trong lớp sẽ là các trường dữ liệu trong đối tượng
 - Mỗi đối tượng có một bộ dữ liệu riêng, không chia sẻ với các đối tượng khác.
 - Các đối tượng hoàn toàn xác định qua “tên” của nó.

2.4 Đối tượng

- 2.4.1 Khai báo đối tượng:

- Cú pháp khai báo:

- Nếu lớp không chứa hàm khởi tạo hoặc có hàm khởi tạo không tham số (hàm khởi tạo ngầm định):

`<tên lớp> <tên đối tượng>;`

- Nếu lớp chứa hàm khởi tạo có tham số:

`<tên lớp> <tên đối tượng>(tham số hàm khởi tạo);`

2.4 Đối tượng

- 2.4.2 Phạm vi, khởi tạo và hủy đối tượng:
 - Phạm vi của đối tượng: là phạm vi mà đối tượng tồn tại kể từ khi đối tượng được tạo ra cho đến khi bị hủy
 - Nếu đối tượng được khai báo bên trong hàm, phạm vi của đối tượng là bên trong hàm đó tính từ khi đối tượng được khai báo-> biến cục bộ
 - Nếu đối tượng được khai báo bên ngoài hàm, phạm vi của đối tượng là toàn bộ chương trình -> biến toàn cục
 - Khi nào đối tượng bị hủy?
 - Đối tượng bị hủy khi hàm chứa đối tượng thực hiện xong hoặc khi chương trình kết thúc.
 - Sử dụng lệnh **delete** với con trỏ đối tượng

2.4 Đối tượng

- 2.4.3 Gọi hàm thành phần của đối tượng
 - Sau khi khai báo đối tượng, có thể gọi hàm (phương thức) của đối tượng ra làm việc
 - Cú pháp:
 - `<tên đối tượng>.<tên hàm thành phần>(<danh sách tham số>);`
 - VD
 - `Point p1;`
 - `P1.init(3,2);`
 - `P1.move(1,1);`
 - `P1.display();`

2.4 Đối tượng

- 2.4.4 Truy xuất và truy nhập dữ liệu thành phần của đối tượng
 - Truy xuất dữ liệu thành phần với hàm bên trong lớp:
 - Hàm bên trong lớp có thể truy cập tới bất kỳ thành phần dữ liệu nào của lớp
 - Truy xuất trực tiếp dữ liệu thành phần bên ngoài lớp:
 - Dữ liệu phải có thuộc tính truy cập là public
 - Cú pháp:
 - <tên đối tượng>.<tên dữ liệu thành phần>
 - Truy xuất gián tiếp dữ liệu thành phần qua hàm thành phần
 - Dữ liệu thành phần có thuộc tính private
 - Hàm thành phần có thuộc tính public

2.5 Nạp chồng hàm

- 2.5.1 Nạp chồng hàm
 - Cho phép nhiều hàm trong một lớp có thể trùng tên nhau nhưng phải khác danh sách tham số (số lượng, kiểu, thứ tự tham số).
 - Khi gọi hàm thành phần từ đối tượng, dựa vào danh sách tham số mà trình biên dịch sẽ chọn hàm thành phần tương ứng để thực hiện

2.5 Nạp chồng hàm

- 2.5 Nạp chồng hàm
 - VD

```
class point{
    int x,y;
public:
    void init(){
        x=0;y=0;
    }
    void init(int a,int b){
        x=a;y=b;
    }
}
```

```
int main{
    point p1;
    p1.init();//x=0,y=0
    p1.init(3,2);//x=3,y=2
    return 0;
}
```

2.5 Nạp chồng hàm

- 2.5.2 Nạp chồng hàm khởi tạo
 - Hàm khởi tạo cũng có thể được nạp chồng
 - Định nghĩa nhiều hàm khởi tạo khác nhau danh sách tham số
 - Cung cấp nhiều cách khởi tạo một đối tượng
 - VD:

2.5 Nạp chồng hàm

- 2.5.2 Nạp chồng toán tử

- Hàm toán tử

- Là các hàm định nghĩa các **phép toán** giữa các đối tượng của lớp.
 - Cũng là một loại hàm: có tham số và giá trị trả về
 - Gần như không có hàm toán tử nào được định nghĩa sẵn (trừ = và []), người dùng phải tự định nghĩa
 - Có thể chồng các hàm toán tử
 - 2 cách để định nghĩa:
 - Hàm thành phần của lớp
 - Hàm bạn (trừ = và [])
 - Cú pháp: **operator<toán tử>**
 - VD: operator+, operator-, operator*=,...

2.5 Nạp chồng hàm

- 2.5.2 Nạp chồng toán tử

- Toán tử 2 ngôi:

- Là các phép toán có 2 số hạng tham gia.

VD: +, -, *, /, %, <<, >>, <, >, <=, >=, ==, !=, &, |, ^, &&, ||

- Nguyên tắc:

- Không định nghĩa toán tử mới
 - Một trong số các toán hạng phải là đối tượng
 - Khi định nghĩa hàm toán tử là một hàm thành phần thì:
 - Hàm chỉ có 1 tham số
 - Đối tượng thực sự gọi hàm toán tử là số hạng trái
 - Tham số của hàm toán tử là số hạng phải

2.5 Nạp chồng hàm

- 2.5.2 Nạp chồng toán tử
 - VD

```
class Point{
    int x,y;
public:
    Point(int ox,int oy){
        x=ox;y=oy;
    }
    Point operator+(Point p){
        Point kq(0,0);
        kq.x=x+p.x;
        kq.y=y+p.y;
        return kq;
    }
};
```

```
int main(){
    Point p1(2,3);

    Point p2(3,4);

    Point c(0,0);
    c=p1+p2; //c.x=5, c.y=7
    //c= p1.operator+(p2)
    return 0;
}
```

2.5 Nạp chồng hàm

- 2.5.2 Nạp chồng toán tử

- Toán tử 1 ngôi:
 - - Là phép toán chỉ có một số hạng, chính là đối tượng gọi hàm.
- VD: *, &, ~, !, ++, --, sizeof()
 - Không có tham số
 - Đối tượng gọi hàm là toán hạng duy nhất
 - Toán tử -- và ++:
 - Tiền tố: `const T & operatorX ();`
 - T: tên lớp
 - X; toán tử -- hoặc ++
 - Hậu tố: `T operatorX (int);`

2.5 Nạp chồng hàm

- 2.5.2 Nạp chồng toán tử

- VD

```
class point{
    int x,y;
public:
    point(int ox,int oy){
        x=ox;y=oy;
    }
    point operator++(){
        x++;y++;
    }
    point &operator--(){
        --x;--y;
    }
}
```

```
int main(){
    point p1(2,3);
    point p2(3,4);
    --p1;
    p2++;
    return 0;
}
```


2.6. Hàm bạn

- Xét hàm thiết lập sao chép ở ví dụ trước:

- `Point (point &p) {`

- `cout<<"sao chep tu p \n";`

- `x=p.x;`  **•ta đang truy cập đến thành phần private của p ở bên ngoài p**

- `y=p.y;` 

- `}` Hàm thành phần có quyền truy cập đến các thành phần private của đối tượng, tham chiếu của đối tượng, con trỏ của đối tượng cùng lớp khi được dùng làm tham số hình thức.

- Vậy một hàm tự do (không thuộc lớp nào) có thể truy cập đến thành phần private của đối tượng khi nó là tham số không?

2.6. Hàm bạn

- Hàm bạn là hàm **không thuộc lớp** nhưng được khai báo là “bạn” của lớp ở bên trong định nghĩa lớp.
- - Hàm bạn có thể truy cập đến các thành phần private của một đối tượng thuộc lớp mà nó kết bạn (giống như hàm thành phần của lớp đó)
- Có nhiều kiểu hàm bạn:
 - + **Hàm bạn là hàm tự do**
 - + Hàm bạn là hàm thành phần của một lớp khác
 - + Hàm bạn của nhiều lớp
 - + Tất cả các hàm thành phần của một lớp là bạn của một lớp khác

2.6. Hàm bạn

```
class Point{
int x,y;
public:
Point(int ox,int oy){
    x=ox;y=oy;
}
friend int compare(Point
p1,Point p2);
};

int compare(Point p1,Point p2){
if(p1.x==p2.x&& p1.y==p2.y)
    return 1;
else return 0;
}
```

•VD:

```
int main(){
point p1(2,3);
...
point p2(3,4);
if(compare(p1,p2)
    cout<<"Trung nhau!";
...

return 0;
}
```